TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN
Học phần Chuyên đề Tốt nghiệp 1 (71ITGR40203)

BÁO CÁO BÀI TẬP 2

Hệ thống chạy được – phiên bản v0.2

SMART CRM – MEKONG MOBILE

Luồng L04: Phân công kỹ thuật viên và lịch hẹn

Track: SE (Kỹ thuật phần mềm)

Sinh viên	Nguyễn Quốc Việt
MSSV	2374802010556
Lớp	261_71ITGR40203_05
Track · Luồng	SE · L04 – Phân công kỹ thuật viên và lịch hẹn
Repo GitHub (public)	`https://github.com/NguyenQuocViet23/smartcrm-2374802010556-phan-cong-ky--thuat-vien-va-lich-hen`
Tag nộp bài	`v0.2 – https://github.com/NguyenQuocViet23/smartcrm-2374802010556-phan-cong-ky--thuat-vien-va-lich-hen/releases/tag/v0.2`
Lệnh lấy mã nguồn	`git clone --branch v0.2 https://github.com/NguyenQuocViet23/smartcrm-2374802010556-phan-cong-ky--thuat-vien-va-lich-hen.git`
Ngày nộp	06/11/2026

TP. Hồ Chí Minh, tháng 11 năm 2026

1. Tóm tắt

Smart CRM quản lý phiếu bảo hành cho chuỗi cửa hàng Mekong Mobile. Luồng L04 gồm hai vai trò. **Quản lý trung tâm** xem phiếu MỚI, nhận gợi ý kỹ thuật viên (cùng trung tâm, tay nghề nhóm sự cố ≥ 3, khối lượng thấp trước), phân công hoặc đổi kỹ thuật viên có lý do, đặt lịch giao/nhận thiết bị không trùng giờ. **Kỹ thuật viên** xem phiếu của mình và cập nhật trạng thái theo vòng đời `MOI → DA_PHAN_CONG → DANG_XU_LY → (CHO_LINH_KIEN) → HOAN_TAT`.

Phiên bản **v0.2** chạy end-to-end trên dữ liệu thật của case study: backend FastAPI ba lớp, PostgreSQL 6 bảng, giao diện React 3 màn hình (M1 Phiếu chờ phân công, M2 Phân công kỹ thuật viên, M3 Phiếu của tôi). So với Buổi 10, Bài tập 2 bổ sung:

- tách ba lớp router → service → repository; service báo lỗi bằng `DomainError` không chứa mã HTTP;
- CRUD đủ cho **Phiếu** và **Lịch hẹn**, thao tác xoá là hủy mềm `DA_HUY` theo QT-13;
- bộ dữ liệu mẫu 462 KB trích từ dữ liệu thật nằm trong repo và lệnh dựng hệ thống `python scripts/setup_all.py`, chạy lại không nhân đôi;
- CSDL không kết nối được → HTTP 503 rõ ràng, giao diện hiện lỗi và nút Thử lại; lỗi ngoài dự kiến → 500 không lộ traceback;
- khoá phiên bản thư viện, `.env.example` đủ biến, nhật ký lỗi 7 mục (`docs/bug-log.md`), khai báo AI (`docs/ai-disclosure.md`).

Đối chiếu với tiêu chí chấm Bài tập 2 (số liệu đo trên máy em và trên bản clone từ GitHub):

Tiêu chí	Kết quả đo được	Minh chứng
C1 – Clone chạy theo README	Clone vào thư mục trống, **không** chép CSV từ LMS, làm đúng các bước README: unit test, `setup_all.py`, backend, frontend chạy với bộ mẫu; 18 TC và e2e pass trên bộ mẫu	Mục 6
C2 – Tách lớp	services + repositories: 0 dòng chứa `APIError|status_code|fastapi|HTTPException`; routers: 0 câu SQL	Hình 2.1
C4 – Bộ kiểm thử	`python -m pytest`: 100 passed, không cần CSDL; integration 7 passed; TC01–TC18: 18 passed	Hình 4.1–4.3
C5 – Chạy lại không nhân đôi	`setup_all.py` lần hai: mọi dòng `NẠP … = 0`, số dòng không đổi; test tự động C5	Hình 4.2, 6.1
C7 – Dữ liệu sai	SĐT `12345` → 400 `INVALID_PHONE` kèm thông báo tiếng Việt	Hình 3.4
Suy giảm có kiểm soát	CSDL không kết nối được → 503 `DATABASE_UNAVAILABLE` sau 3 giây; M1 hiện lỗi + Thử lại	Hình 5.5
S1 – CRUD ≥ 2 thực thể	Phiếu và Lịch hẹn: tạo · xem · sửa · hủy mềm	Mục 3
S3 – Migration + seed	`db/migrations/001_create_tables.sql`, loader idempotent (`ON CONFLICT`)	Mục 6
S4 – ≥ 3 màn hình, đủ trạng thái	M1, M2, M3; có dữ liệu · đang tải · rỗng · lỗi	Mục 5

2. Kiến trúc đã hiện thực

Giao diện React gọi backend qua REST/JSON có token. Backend tách ba lớp, mỗi lớp chỉ gọi lớp ngay dưới. Quy tắc nghiệp vụ là hàm thuần nên kiểm thử được mà không cần CSDL. Sơ đồ dưới đây ghi đúng tên thư mục, tệp trong mã nguồn:

Giao diện – React + Vite (`src/frontend/src`)
M1 `PendingTicketsPage`, M2 `AssignTechnicianPage`, M3 `MyTicketsPage`; `api/client.js` gắn token, đổi lỗi API thành thông báo; `DataState` hiển thị đang tải · rỗng · lỗi + Thử lại.

↓ *HTTP + JSON (Bearer token)*

Router – `app/routers/`
`tickets.py`, `appointments.py`, `technicians.py`, `auth.py`, `health.py`: nhận request, kiểm schema Pydantic, lấy người dùng hiện tại, gọi service, trả JSON. **Không SQL, không quy tắc nghiệp vụ.**

↓ *gọi hàm Python; lỗi nghiệp vụ = DomainError(kind, code, message)*

Service – `app/services/`
`ticket_service`, `appointment_service`: điều phối giao dịch, phân quyền phạm vi; `ticket_rules`, `appointment_rules`: hàm thuần (vòng đời trạng thái, tay nghề ≥ 3, SĐT QT-02, giờ làm việc, khung giờ trống). **Không biết HTTP.**

↓ *gọi repository trong giao dịch (`read_connection()` / `transaction()`)*

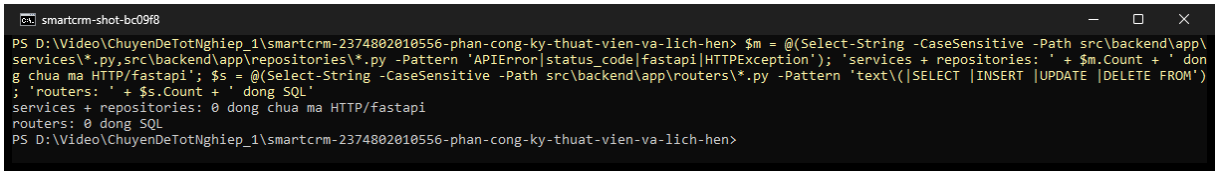
Repository – `app/repositories/`
`ticket_repository`, `appointment_repository`, `health_repository`: toàn bộ SQL tham số hoá; vi phạm ràng buộc chống trùng lịch (SQLSTATE 23P01) → `OverlapViolation`.

↓ *SQL*

PostgreSQL 16+ (`db/migrations/001_create_tables.sql`)
6 bảng `service_center`, `technician`, `technician_skill`, `ticket`, `ticket_status_log`, `appointment`; khoá ngoại, CHECK, `EXCLUDE USING gist` chống trùng lịch của một kỹ thuật viên.

Chỉ `app/main.py` biết mã HTTP: bảng `HTTP_STATUS` ánh xạ `ErrorKind` → mã trạng thái và các exception handler (`handle_domain_error`, `handle_database_unavailable`, `handle_unexpected_error`). Phản hồi lỗi luôn có dạng `{"error": "<MÃ>", "message": "<thông báo>"}`.

Loại lỗi trong service / hạ tầng	HTTP	Ví dụ mã lỗi thật trong mã nguồn
`ErrorKind.INVALID`	400	`INVALID_PHONE`, `TECHNICIAN_NOT_ELIGIBLE`, `OUTSIDE_WORKING_HOURS`, `REASON_REQUIRED`
`ErrorKind.UNAUTHENTICATED` / `FORBIDDEN`	401 / 403	token hết hạn; xem phiếu của trung tâm hoặc kỹ thuật viên khác
`ErrorKind.NOT_FOUND`	404	`TICKET_NOT_FOUND`, `APPOINTMENT_NOT_FOUND`
`ErrorKind.CONFLICT`	409	`INVALID_STATUS_TRANSITION`, `APPOINTMENT_OVERLAP`, `TICKET_NOT_CANCELLABLE`
`OperationalError` / `InterfaceError` (CSDL tắt)	503	`DATABASE_UNAVAILABLE` – thời gian chờ kết nối `DB_CONNECT_TIMEOUT` = 3 s
`Exception` ngoài dự kiến	500	`INTERNAL_ERROR` – ghi log bằng `logging`, không trả traceback



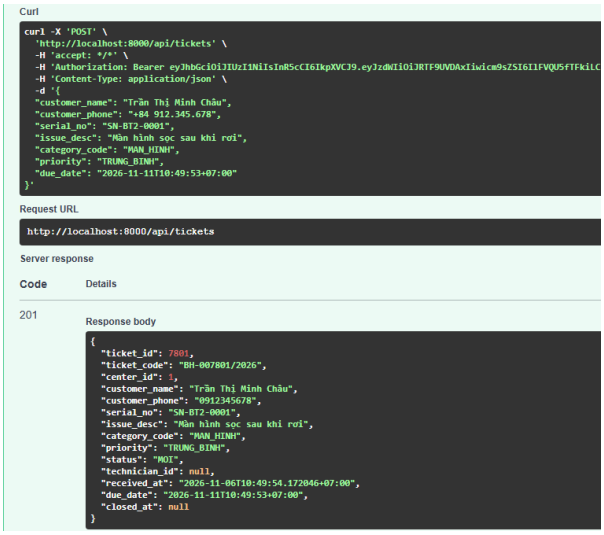
Hình 2.1 – Kiểm tra tách lớp (C2) trên mã nguồn thật: service + repository có 0 dòng chứa mã HTTP/fastapi, router có 0 câu SQL

3. Module cốt lõi

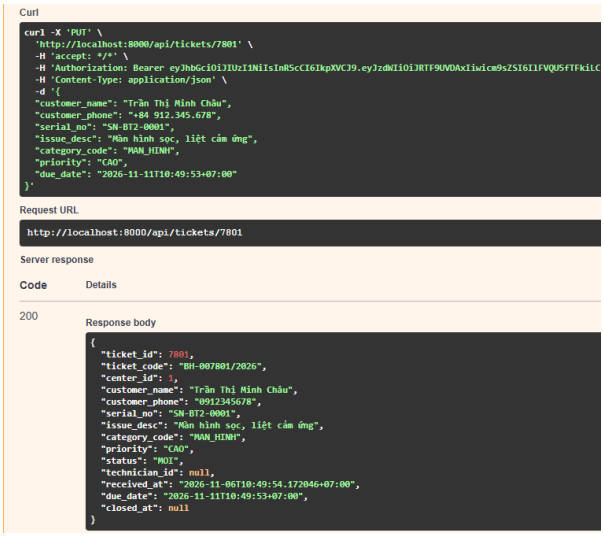
Luồng chính của L04 (gợi ý → phân công → đặt lịch → cập nhật trạng thái) đã chạy từ Buổi 10 và được kiểm lại bằng TC01–TC18 sau khi tách lớp (mục 4). Bài tập 2 bổ sung CRUD đủ cho hai thực thể; thao tác xoá là hủy mềm vì QT-13 cấm xoá vật lý phiếu và lịch sử:

Thực thể	Create	Read	Update	Delete (hủy mềm)
Phiếu (Quản lý, trong trung tâm của mình)	`POST /api/tickets` → MỚI, chuẩn hoá SĐT, sinh mã `BH-<id>/<năm>`, ghi log	`GET /api/tickets`, `GET /api/tickets/{id}`	`PUT /api/tickets/{id}` khi còn MỚI	`DELETE /api/tickets/{id}` → `DA_HUY`, ghi log, khi còn MỚI
Lịch hẹn (Quản lý; KTV xem lịch của mình)	`POST /api/appointments`	`GET /api/appointments`, `GET /api/appointments/{id}`	`PUT /api/appointments/{id}`: kiểm lại giờ làm việc và chống trùng	`DELETE /api/appointments/{id}` → `DA_HUY`, giải phóng khung giờ

Ảnh dưới chụp trình duyệt thật (Swagger UI của backend đang chạy trên bản sao dữ liệu `smartcrm_work_test`), đã cắt phần request (curl) và phản hồi; ảnh đầy đủ ở `docs/bt2-evidence/BT2_01…06`.



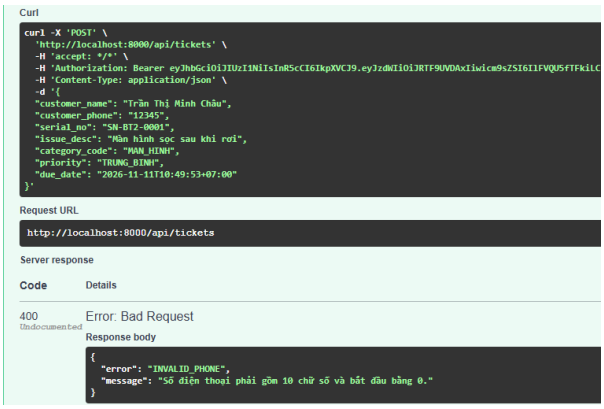
Hình 3.1 – Tạo phiếu: 201, `status` *= MOI, SĐT* `+84 912.345.678` *chuẩn hoá thành* `0912345678`



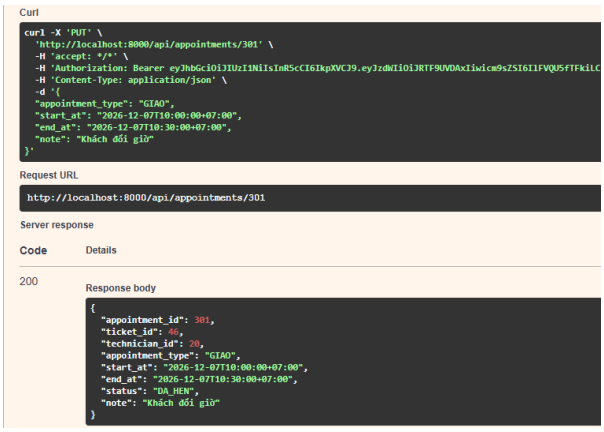
Hình 3.2 – Sửa phiếu 7801 khi còn MỚI: 200, `priority` *= CAO, mô tả mới*



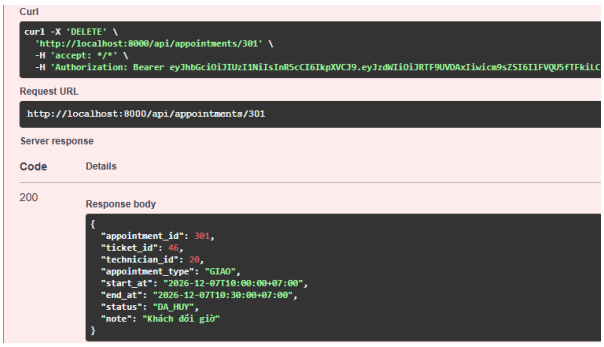
Hình 3.3 – Hủy mềm phiếu 7801 kèm lý do: 200, `status` *= DA_HUY, có* `closed_at`



Hình 3.4 – Dữ liệu sai (C7): SĐT `12345` *→ 400* `INVALID_PHONE`



Hình 3.5 – Đổi giờ lịch hẹn 301 sang 10:00–10:30, loại GIAO: 200



Hình 3.6 – Hủy mềm lịch hẹn 301: 200, `status` *= DA_HUY*

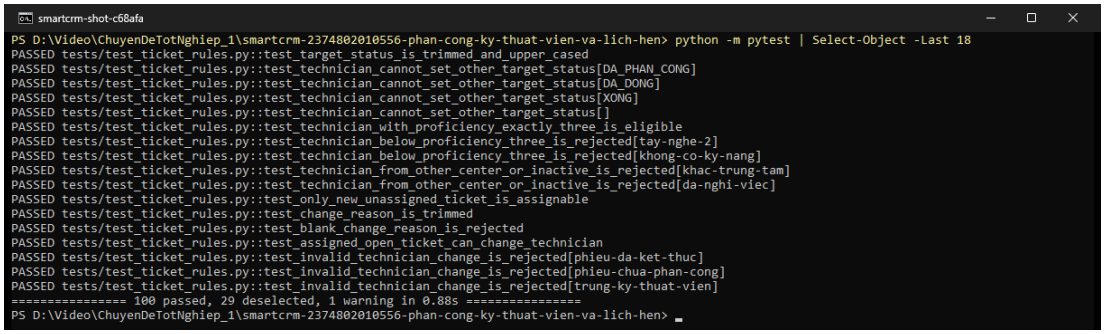
4. Bộ kiểm thử

Unit test chạy bằng **một lệnh** `python -m pytest`, không cần CSDL hay frontend (`pytest.ini` loại các nhóm cần hạ tầng). Các nhóm còn lại chạy trên PostgreSQL thật với database có tên kết thúc bằng `_test`; test không bao giờ ghi vào `smartcrm`.

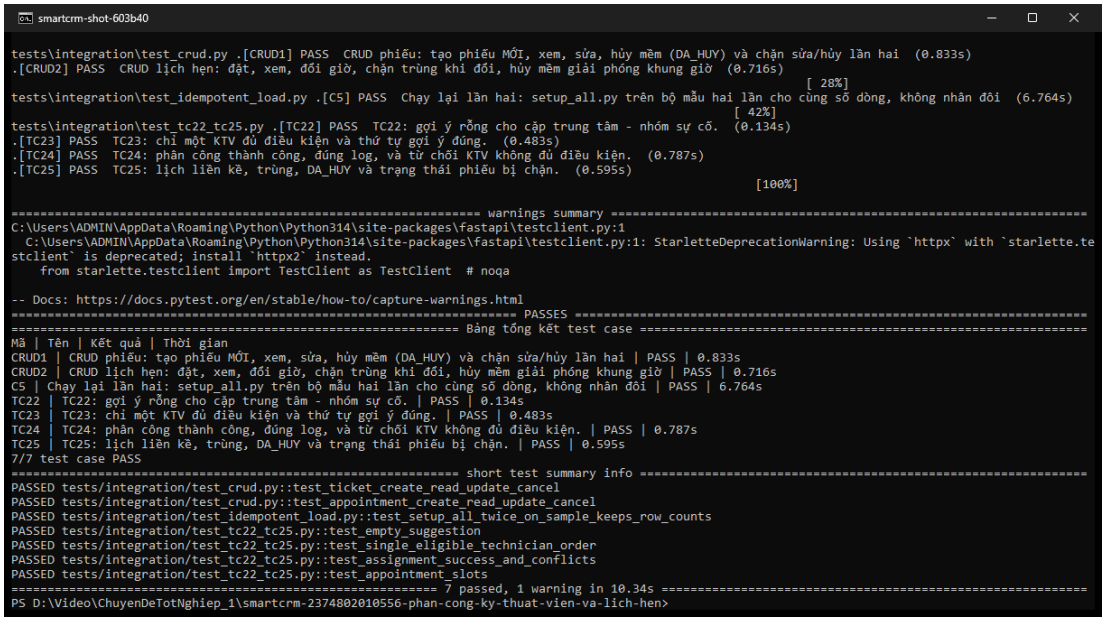
Nhóm	Lệnh	Cần gì	Kết quả
Unit (quy tắc nghiệp vụ, ánh xạ lỗi, API qua TestClient)	`python -m pytest`	Không cần CSDL	100 passed (0,88 s)
Integration (CRUD, C5, TC22–TC25)	`python -m pytest tests\integration -m integration`	PostgreSQL, `setup_all.py --with-test-db`	7 passed (10,3 s)
Hệ thống TC01–TC18 (nghiệp vụ + giao diện + e2e)	`python -m pytest -m system`	PostgreSQL + frontend	18 passed (4 phút 22 giây)
End-to-end luồng chính (có tắt backend)	`python -m pytest tests\e2e -m e2e`	như trên	1 passed (23,8 s)
Giao diện với API giả lập (TC19–TC21)	`python -m pytest tests\ui -m ui`	Frontend	3 passed (4,0 s)

Unit test chia theo ba loại – luồng đúng, giá trị biên, dữ liệu sai:

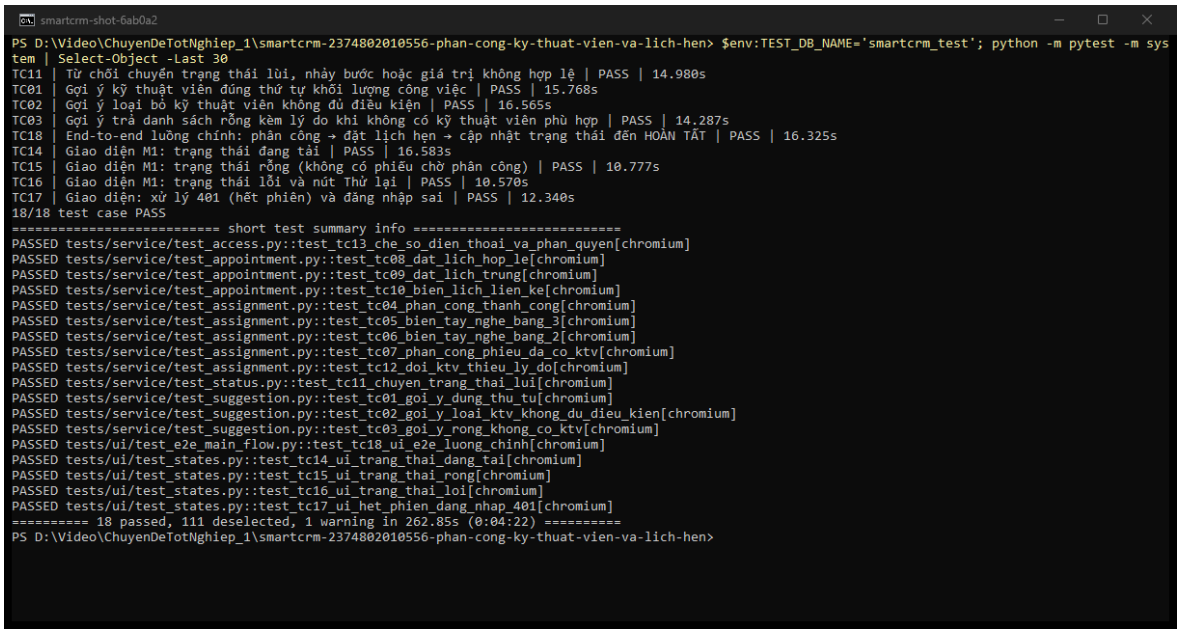
Tệp test	Số test	Đúng	Biên	Sai
`test_ticket_rules.py`	33	DA_PHAN_CONG → DANG_XU_LY; đổi KTV khi phiếu đang mở	tay nghề **đúng bằng 3** được nhận	tay nghề 2, khác trung tâm, đã nghỉ việc, chuyển lùi, lý do rỗng
`test_crud_rules.py`	23	`0909 123 456` → `0909123456`; sửa/hủy phiếu MỚI	hạn cam kết sau hiện tại **1 giây** được nhận; bằng hiện tại bị từ chối	SĐT `12345`; hủy phiếu đã phân công; hủy lịch đã hủy
`test_appointment_validation.py`	3	08:00–08:30 ngày làm việc	lịch kề nhau kết thúc đúng **18:00**	lịch ngày Chủ nhật
`test_error_handling.py`	11	mỗi `ErrorKind` có đúng một mã HTTP	CSDL tắt → 503, có `connect_timeout`	lỗi ngoài dự kiến → 500 không lộ traceback; payload sai bị chặn trước service
Còn lại (nạp dữ liệu, xác thực, guard DB)	30	token hợp lệ, chuẩn hoá dòng CSV	trạng thái có dấu/không dấu	mật khẩu sai, tên DB không kết thúc `_test`



Hình 4.1 – `python -m pytest`*: 100 passed, 29 deselected (nhóm cần CSDL/trình duyệt chạy bằng lệnh riêng)*



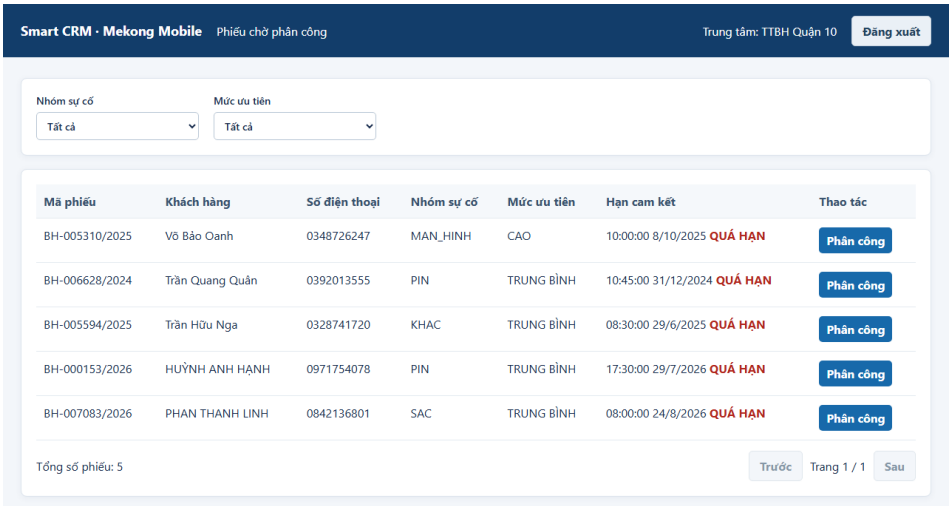
Hình 4.2 – Integration trên PostgreSQL: CRUD1, CRUD2, C5 (setup_all.py hai lần trên bộ mẫu cho cùng số dòng), TC22–TC25 – 7 passed



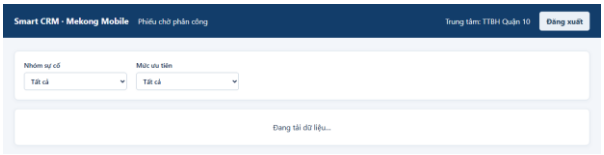
Hình 4.3 – 18 test case TC01–TC18 trên backend, giao diện và dữ liệu thật: 18 passed

5. Trình bày kết quả

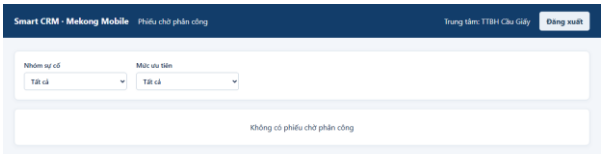
Ba màn hình M1, M2, M3 dùng chung thành phần `DataState`, nên mọi màn hình đều có đủ trạng thái: có dữ liệu, đang tải, rỗng và lỗi kèm nút Thử lại. Ảnh chụp bằng Playwright trên giao diện React thật (TC14, TC15, TC16, TC18 trong `docs/test-evidence/`), cắt bỏ phần nền trống bên dưới.



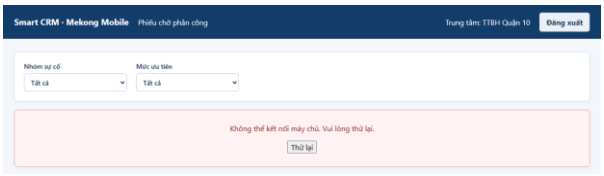
Hình 5.1 – M1 có dữ liệu: 5 phiếu MỚI của TTBH Quận 10, hạn quá hạn tô đỏ, nút Phân công (TC18 bước 1)



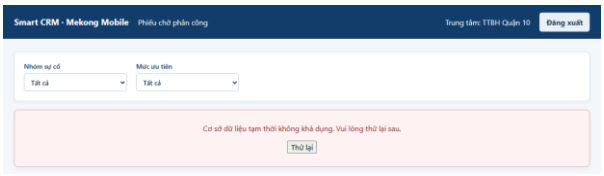
Hình 5.2 – Đang tải: "Đang tải dữ liệu…" (TC14, giữ request 3 giây)



Hình 5.3 – Rỗng: "Không có phiếu chờ phân công" (TC15, TTBH Cầu Giấy)



Hình 5.4 – Lỗi mất kết nối backend: thông báo + nút Thử lại (TC16)



Hình 5.5 – Suy giảm có kiểm soát: CSDL không kết nối được → API 503, M1 báo "Cơ sở dữ liệu tạm thời không khả dụng" + Thử lại

Hình 5.5 được chụp khi backend chạy với `DB_PORT=1` (không có PostgreSQL). Trước khi sửa BUG-07, API treo hơn 2 phút; sau khi đặt `connect_timeout`, API trả 503 sau 3,0 giây và giao diện báo lỗi ngay. Bật lại CSDL và bấm Thử lại thì danh sách hiện lại.

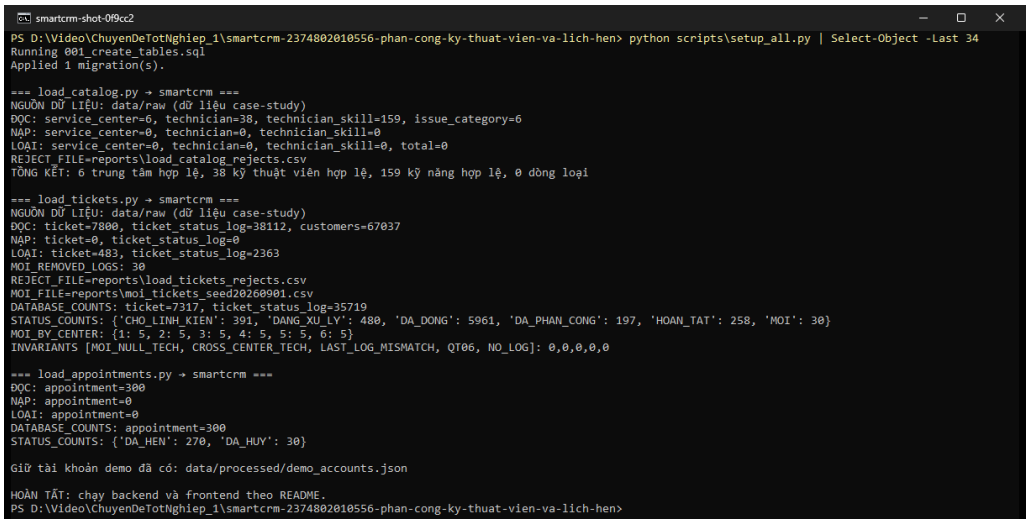
6. Cách dựng lại

Cần cài sẵn Git, Python 3.12+, Node.js 22+, PostgreSQL 16+ đang chạy. Không cần tải dữ liệu: repo có bộ mẫu `data/sample/` (6 trung tâm, 38 kỹ thuật viên, 159 tay nghề, 939 phiếu, 3.237 dòng lịch sử, 300 lịch hẹn) trích từ dữ liệu thật. Nếu chép bộ 7 CSV đầy đủ từ LMS vào `data/raw/` thì loader tự dùng bộ đầy đủ. Trên Windows nên clone vào thư mục có đường dẫn ngắn (BUG-04).

```
# Cửa sổ PowerShell 1 – tại thư mục muốn đặt repo
git clone --branch v0.2 https://github.com/NguyenQuocViet23/smartcrm-2374802010556-phan-cong-ky--thuat-vien-va-lich-hen.git smartcrm
Set-Location smartcrm
Copy-Item .env.example .env
notepad .env                  # sửa DB_USER, DB_PASSWORD, SECRET_KEY (>= 32 ký tự), DEMO_PASSWORD
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
python -m pytest              # 100 passed
python scripts\setup_all.py   # tạo DB -> migrate -> nạp dữ liệu -> tài khoản demo
python -m uvicorn app.main:app --app-dir src\backend        # http://localhost:8000/docs
# Cửa sổ PowerShell 2 – tại thư mục repo
Set-Location src\frontend
npm ci
npm run dev                   # http://localhost:5173 – đăng nhập QL_TT01 / DEMO_PASSWORD
```

Kết quả mong đợi của `setup_all.py` với bộ mẫu: `NGUỒN DỮ LIỆU: data/sample`, `ticket=939`, `ticket_status_log=3237`, `appointment=300`, 30 phiếu MOI (mỗi trung tâm 5), các truy vấn bất biến

đều 0. Chạy lại lần hai: mọi dòng `NẠP … = 0`, số dòng không đổi. Tài khoản: `QL_TT01`…`QL_TT06` (Quản lý), `KT001`…`KT038` (Kỹ thuật viên, trừ `KT001`, `KT011`, `KT031` đã nghỉ việc). Chạy test tích hợp: `python scripts\setup_all.py --with-test-db` một lần rồi dùng các lệnh ở mục 4. Có thêm cách chạy bằng Docker Compose ở README mục 3.3.



Hình 6.1 – Chạy `setup_all.py` lần hai trên máy em (có bộ CSV đầy đủ trong `data/raw`): mọi dòng NẠP = 0, số dòng giữ nguyên, bất biến 0,0,0,0,0

Kiểm chứng C1: đẩy mã lên GitHub, clone vào thư mục trống mới, **không** chép CSV từ LMS, làm đúng từng bước README: unit, `setup_all.py` (hai lần), integration 7 passed, TC01–TC18 18 passed, e2e passed trên bộ mẫu. Lỗi phát hiện ở các lần clone thử được ghi trong `docs/bug-log.md` (BUG-03, BUG-04) và đã sửa trước khi gắn tag.

7. Thay đổi so với BT1

Chi tiết từng thay đổi (vấn đề · quyết định SỬA/GIỮ · lý do · ảnh hưởng) ở `docs/thay-doi-thiet-ke.md`. Lược đồ 6 bảng của BT1 được giữ nguyên; các thay đổi chính:

Mã	Thay đổi	Lý do
TD-01, 02	Thêm API đăng nhập, `/api/auth/me`, chi tiết phiếu, lọc + phân trang; tài khoản demo lưu ở JSON (PBKDF2), không thêm bảng	Ba màn hình cần API thật; giữ ERD 6 bảng
TD-03, 07, 11	Chuẩn hoá dữ liệu nạp; loại 483 phiếu (6,2%) không có SĐT hợp lệ cùng 2.363 dòng log	SĐT NOT NULL, QT-02; không tạo dữ liệu giả
TD-08, 09, 10	Seed 30 phiếu MOI (5/trung tâm, seed 20260901); sinh 300 lịch hẹn bằng `gen_appointments.py`; không đặt lịch cho phiếu HOAN_TAT	`gen_sample.py` sinh trạng thái ngoài schema; tuân SRS UC6
TD-05, 15	Không làm UC8 (COULD), màn hình thứ tư; nút hủy lịch trên giao diện hoãn (đã có API)	Bám phạm vi L04 và wireframe M1–M3
TD-12, 16	Test ghi chạy trên `smartcrm_work_test` tạo lại từ `smartcrm_test`; TC01–TC18 chạy trên hệ thống thật, chỉ ảnh trình duyệt thật	Không làm bẩn dữ liệu thật; lặp lại được
TD-13	M3 thêm bộ lọc trạng thái, KTV vẫn xem phiếu đã đóng/hủy **của mình**	Dữ liệu thật tới 443 phiếu/KTV
TD-14, 18	Quy tắc thành hàm thuần; tách router/service/repository; `DomainError` không có mã HTTP; xử lý 503/500	Tiêu chí C2; test không cần CSDL
TD-17	CRUD đủ cho Phiếu và Lịch hẹn, xoá = hủy mềm `DA_HUY`	Tiêu chí S1; QT-13
TD-19	Bộ mẫu `data/sample/`, `setup_all.py`, khoá phiên bản `requirements*.txt`, `.env.example` đủ biến; xoá script seed cũ	Tiêu chí C1, C5

Phụ lục – Khai báo sử dụng công cụ AI

Bản đầy đủ: `docs/ai-disclosure.md`. Mọi phần có AI hỗ trợ đều được em đọc lại, chạy lại và giải thích được; kết quả ghi trong báo cáo là kết quả chạy thật.

Công cụ	Việc AI hỗ trợ	Tệp / hàm liên quan	Cách em kiểm chứng
Claude (chat)	BT1: bản nháp SRS, use case, kiến trúc, ERD, wireframe; viết prompt giao việc cho Copilot	`docs/srs.md`, `docs/*.drawio`, `db/schema.sql`	Đối chiếu từng QT-xx với case study; chạy `schema.sql` trên PostgreSQL thật; mở từng `.drawio` kiểm tác nhân, ERD
GitHub Copilot	Buổi 8–10: giao diện M1–M3, API ban đầu, script nạp dữ liệu, sinh lịch hẹn	`src/frontend/src/**`, `scripts/load_*.py`, `gen_appointments.py`	`npm run build`; đếm số dòng 6 bảng bằng SQL; truy vấn bất biến trả 0 dòng; đọc `git diff` trước khi commit
Claude Code	Tách 3 lớp backend, lỗi miền, ánh xạ lỗi → HTTP	`app/routers/*`, `app/services/*_service.py`, `app/repositories/*`, `app/domain/errors.py`, `app/main.py`	Test cũ (18 TC, e2e, integration) vẫn pass sau khi tách; lệnh kiểm tách lớp trả 0 dòng (Hình 2.1)
Claude Code	Quy tắc nghiệp vụ thuần và unit test 3 loại	`ticket_rules.py`, `appointment_rules.py`; `tests/test_*_rules.py`	`python -m pytest` 100 passed; đọc từng test, đối chiếu QT-02, QT-06, QT-08, QT-13
Claude Code	CRUD phiếu và lịch hẹn (hủy mềm)	`ticket_service.create/update/cancel_ticket`, `appointment_service.update/cancel_appointment`, `tests/integration/test_crud.py`	Integration kiểm từng bước bằng SQL; thử lại bằng Swagger (Hình 3.1–3.6)
Claude Code	Xử lý CSDL tắt (503) và lỗi ngoài dự kiến (500)	`main.py`: `handle_database_unavailable`, `handle_unexpected_error`; `database.py`: `get_engine`	Cho backend trỏ tới cổng không có PostgreSQL rồi gọi API; unit test kiểm phản hồi không lộ lỗi thô
Claude Code	Bộ dữ liệu mẫu, dựng hệ thống một lệnh	`make_sample_data.py`, `data_source.py`, `setup_all.py`, `test_idempotent_load.py`	Chạy `setup_all.py` hai lần, số dòng không đổi; 30 phiếu MOI của bộ mẫu trùng dữ liệu đầy đủ
Claude Code	TC01–TC18 trên hệ thống thật, e2e, ảnh minh chứng	`tests/service/*`, `tests/ui/*`, `tests/e2e/*`, `tests/real_system.py`	`pytest -m system` 18 passed; mở từng ảnh; đưa lại mã lỗi cũ để chắc test bắt được lỗi
Claude Code	README, khoá phiên bản, `.env.example`, nhật ký lỗi, báo cáo này	`README.md`, `requirements*.txt`, `docs/bug-log.md`, `docs/thay-doi-thiet-ke.md`	Clone repo vào thư mục trống và làm đúng từng bước README; quét toàn bộ lịch sử Git không có secret

Quyết định do em đưa ra khi rà soát kết quả AI:

- Không ẩn phiếu đã đóng/đã hủy ở M3; kỹ thuật viên chỉ xem phiếu của chính mình, thêm bộ lọc (TD-13).
- Chọn CRUD cho Phiếu và Lịch hẹn, xoá là hủy mềm theo QT-13; đưa bộ mẫu nhỏ trích từ dữ liệu thật vào repo (TD-17, TD-19).
- Bỏ ảnh minh chứng vẽ bằng thư viện, chỉ dùng ảnh chụp trình duyệt và cửa sổ console thật; giữ nguyên thư mục `docs/diagrams/`.
- Yêu cầu kiểm chứng lại bằng bản clone từ GitHub trước khi gắn tag `v0.2`.

Em đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp, kể cả phần có AI hỗ trợ, và giải thích được từng hàm khi được hỏi.

Sinh viên: Nguyễn Quốc Việt – MSSV 2374802010556